JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class ConcurrentHashMap.KeySetView<K,V>

java.lang.Object
  java.util.concurrent.ConcurrentHashMap.KeySetView<K,V>

All Implemented Interfaces:

Serializable, Iterable<K>, Collection<K>, Set<K>

Enclosing class:

ConcurrentHashMap<K,V>

```
public static class ConcurrentHashMap.KeySetView<K,V>
extends Object
implements Set<K>, Serializable
```

A view of a ConcurrentHashMap as a Set of keys, in which additions may optionally be enabled by mapping to a common value. This class cannot be directly instantiated. See keySet(), keySet(V), newKeySet(), newKeySet(int).

Since:

1.8

See Also:

Serialized Form

Method Summary

Concrete Methods

Modifier and Type	Method	Description
boolean	add(K e)	Adds the specified key to this set view by mapping

		the key to the default mapped value in the backing map, if defined.
boolean	**addAll**(**Collection**<? extends **K**> c)	Adds all of the elements in the specified collection to this set, as if by calling **add(K)** on each one.
void	**clear**()	Removes all of the elements from this view, by removing all the mappings from the map backing this view.
boolean	**contains**(**Object** o)	Returns `true` if this collection contains the specified element.
boolean	**containsAll**(**Collection**<?> c)	Returns `true` if this collection contains all of the elements in the specified collection.
boolean	**equals**(**Object** o)	Indicates whether some other object is "equal to" this one.
void	**forEach**(**Consumer**<? super **K**> action)	Performs the given action for each element of the `Iterable` until all elements have been processed or the action throws an exception.

ConcurrentHashMap<K,V>	getMap()	Returns the map backing this view.
V	getMappedValue()	Returns the default mapped value for additions, or null if additions are not supported.
int	hashCode()	Returns a hash code value for the object.
boolean	isEmpty()	Returns true if this collection contains no elements.
Iterator<K>	iterator()	Returns an iterator over the elements in this collection.
boolean	remove(Object o)	Removes the key from this map view, by removing the key (and its corresponding value) from the backing map.
boolean	removeAll(Collection<?> c)	Removes all of this collection's elements that are also contained in the specified collection (optional operation).
boolean	retainAll(Collection<?> c)	Retains only the elements in this collection that are contained in the specified collection (optional operation).

int	**size**()	Returns the number of elements in this collection.
Spliterator<**K**>	**spliterator**()	Creates a `Spliterator` over the elements in this set.
Object[]	**toArray**()	Returns an array containing all of the elements in this collection.
<T> T[]	**toArray**(T[] a)	Returns an array containing all of the elements in this collection; the runtime type of the returned array is that of the specified array.
String	**toString**()	Returns a string representation of this collection.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Methods inherited from interface java.util.Set

clear, containsAll, isEmpty, removeAll, retainAll, size, toArray, toArray

Methods inherited from interface java.util.Collection

parallelStream, removeIf, stream

Method Detail

getMappedValue

```
public V getMappedValue()
```

Returns the default mapped value for additions, or `null` if additions are not supported.

Returns:

the default mapped value for additions, or null if not supported

contains

```
public boolean contains(Object o)
```

Returns `true` if this collection contains the specified element. More formally, returns `true` if and only if this collection contains at least one element `e` such that `(o==null ? e==null : o.equals(e))`.

Specified by:

`contains` in interface `Collection<K>`

Specified by:

`contains` in interface `Set<K>`

Parameters:

o - element whose presence in this collection is to be tested

Returns:

true if this collection contains the specified element

Throws:

`NullPointerException` - if the specified key is null

remove

```
public boolean remove(Object o)
```

Removes the key from this map view, by removing the key (and its corresponding value) from the backing map. This method does nothing if the key is not in the map.

Specified by:

`remove` in interface `Collection<K>`

Specified by:

`remove` in interface `Set<K>`

Parameters:

o - the key to be removed from the backing map

Returns:

true if the backing map contained the specified key

Throws:

NullPointerException - if the specified key is null

iterator

public Iterator<K> iterator()

Returns an iterator over the elements in this collection.

The returned iterator is *weakly consistent*.

Specified by:

iterator in interface Iterable<K>

Specified by:

iterator in interface Collection<K>

Specified by:

iterator in interface Set<K>

Returns:

an iterator over the keys of the backing map

add

public boolean add(K e)

Adds the specified key to this set view by mapping the key to the default mapped value in the backing map, if defined.

Specified by:

add in interface Collection<K>

Specified by:

add in interface Set<K>

Parameters:

e - key to be added

Returns:

true if this set changed as a result of the call

Throws:

NullPointerException - if the specified key is null

UnsupportedOperationException - if no default mapped value for additions was provided

addAll

public boolean addAll(Collection<? extends K> c)

Adds all of the elements in the specified collection to this set, as if by calling add(K) on each one.

Specified by:

addAll in interface Collection<K>

Specified by:

addAll in interface Set<K>

Parameters:

c - the elements to be inserted into this set

Returns:

true if this set changed as a result of the call

Throws:

NullPointerException - if the collection or any of its elements are null

UnsupportedOperationException - if no default mapped value for additions was provided

See Also:

Set.add(Object)

hashCode

public int hashCode()

Description copied from class: Object

Returns a hash code value for the object. This method is supported for the benefit of hash tables such as those provided by HashMap.

The general contract of hashCode is:

- Whenever it is invoked on the same object more than once during an execution of a Java application, the hashCode method must consistently return the same integer, provided no information used in equals comparisons on the object is modified. This integer need not remain consistent from one execution of an application to another execution of the same application.

- If two objects are equal according to the `equals(Object)` method, then calling the `hashCode` method on each of the two objects must produce the same integer result.
- It is *not* required that if two objects are unequal according to the `Object.equals(java.lang.Object)` method, then calling the `hashCode` method on each of the two objects must produce distinct integer results. However, the programmer should be aware that producing distinct integer results for unequal objects may improve the performance of hash tables.

As much as is reasonably practical, the hashCode method defined by class `Object` does return distinct integers for distinct objects. (This is typically implemented by converting the internal address of the object into an integer, but this implementation technique is not required by the Java™ programming language.)

Specified by:

`hashCode` in interface `Collection<K>`

Specified by:

`hashCode` in interface `Set<K>`

Overrides:

`hashCode` in class `Object`

Returns:

a hash code value for this object.

See Also:

`Object.equals(java.lang.Object)`, `System.identityHashCode(java.lang.Object)`

equals

```
public boolean equals(Object o)
```

Description copied from class: `Object`

Indicates whether some other object is "equal to" this one.

The `equals` method implements an equivalence relation on non-null object references:

- It is *reflexive*: for any non-null reference value `x`, `x.equals(x)` should return `true`.
- It is *symmetric*: for any non-null reference values `x` and `y`, `x.equals(y)` should return `true` if and only if `y.equals(x)` returns `true`.
- It is *transitive*: for any non-null reference values `x`, `y`, and `z`, if `x.equals(y)` returns `true` and `y.equals(z)` returns `true`, then `x.equals(z)` should return `true`.

- It is *consistent*: for any non-null reference values `x` and `y`, multiple invocations of `x.equals(y)` consistently return `true` or consistently return `false`, provided no information used in `equals` comparisons on the objects is modified.
- For any non-null reference value `x`, `x.equals(null)` should return `false`.

The `equals` method for class `Object` implements the most discriminating possible equivalence relation on objects; that is, for any non-null reference values `x` and `y`, this method returns `true` if and only if `x` and `y` refer to the same object (`x == y` has the value `true`).

Note that it is generally necessary to override the `hashCode` method whenever this method is overridden, so as to maintain the general contract for the `hashCode` method, which states that equal objects must have equal hash codes.

Specified by:

`equals` in interface `Collection<K>`

Specified by:

`equals` in interface `Set<K>`

Overrides:

`equals` in class `Object`

Parameters:

`o - the reference object with which to compare.`

Returns:

`true if this object is the same as the obj argument; false otherwise.`

See Also:

`Object.hashCode()`, `HashMap`

spliterator

`public Spliterator<K> spliterator()`

Description copied from interface: `Set`

Creates a `Spliterator` over the elements in this set.

The `Spliterator` reports `Spliterator.DISTINCT`. Implementations should document the reporting of additional characteristic values.

Specified by:

`spliterator` in interface `Iterable<K>`

Specified by:

`spliterator` in interface `Collection<K>`

Specified by:

`spliterator` in interface `Set<K>`

Returns:

a Spliterator over the elements in this set

forEach

```
public void forEach(Consumer<? super K> action)
```

Description copied from interface: `Iterable`

Performs the given action for each element of the `Iterable` until all elements have been processed or the action throws an exception. Unless otherwise specified by the implementing class, actions are performed in the order of iteration (if an iteration order is specified). Exceptions thrown by the action are relayed to the caller.

Specified by:

`forEach` in interface `Iterable<K>`

Parameters:

`action` - The action to be performed for each element

getMap

```
public ConcurrentHashMap<K,V> getMap()
```

Returns the map backing this view.

Returns:

the map backing this view

clear

```
public final void clear()
```

Removes all of the elements from this view, by removing all the mappings from the map backing this view.

Specified by:

`clear` in interface `Collection<E>`

size

```
public final int size()
```

Description copied from interface: Collection

Returns the number of elements in this collection. If this collection contains more than `Integer.MAX_VALUE` elements, returns `Integer.MAX_VALUE`.

Specified by:

`size` in interface `Collection<E>`

Returns:

the number of elements in this collection

isEmpty

```
public final boolean isEmpty()
```

Description copied from interface: Collection

Returns `true` if this collection contains no elements.

Specified by:

`isEmpty` in interface `Collection<E>`

Returns:

true if this collection contains no elements

toArray

```
public final Object[] toArray()
```

Description copied from interface: Collection

Returns an array containing all of the elements in this collection. If this collection makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

The returned array will be "safe" in that no references to it are maintained by this collection. (In other words, this method must allocate a new array even if this collection is backed by an array). The caller is thus free to modify the returned array.

This method acts as bridge between array-based and collection-based APIs.

Specified by:

`toArray` in interface `Collection<E>`

Returns:

an array containing all of the elements in this collection

toArray

```
public final <T> T[] toArray(T[] a)
```

Description copied from interface: `Collection`

Returns an array containing all of the elements in this collection; the runtime type of the returned array is that of the specified array. If the collection fits in the specified array, it is returned therein. Otherwise, a new array is allocated with the runtime type of the specified array and the size of this collection.

If this collection fits in the specified array with room to spare (i.e., the array has more elements than this collection), the element in the array immediately following the end of the collection is set to `null`. (This is useful in determining the length of this collection *only* if the caller knows that this collection does not contain any `null` elements.)

If this collection makes any guarantees as to what order its elements are returned by its iterator, this method must return the elements in the same order.

Like the `Collection.toArray()` method, this method acts as bridge between array-based and collection-based APIs. Further, this method allows precise control over the runtime type of the output array, and may, under certain circumstances, be used to save allocation costs.

Suppose `x` is a collection known to contain only strings. The following code can be used to dump the collection into a newly allocated array of `String`:

```
String[] y = x.toArray(new String[0]);
```

Note that `toArray(new Object[0])` is identical in function to `toArray()`.

Specified by:

`toArray` in interface `Collection<E>`

Type Parameters:

`T` - the runtime type of the array to contain the collection

Parameters:

`a` - the array into which the elements of this collection are to be stored, if it is big enough; otherwise, a new array of the same runtime type is allocated for this purpose.

Returns:

an array containing all of the elements in this collection

toString

```
public final String toString()
```

Returns a string representation of this collection. The string representation consists of the string representations of the collection's elements in the order they are returned by its iterator, enclosed in square brackets ("[]"). Adjacent elements are separated by the characters ", " (comma and space). Elements are converted to strings as by `String.valueOf(Object)`.

Overrides:

`toString` in class `Object`

Returns:

a string representation of this collection

containsAll

```
public final boolean containsAll(Collection<?> c)
```

Description copied from interface: `Collection`

Returns `true` if this collection contains all of the elements in the specified collection.

Specified by:

`containsAll` in interface `Collection<E>`

Parameters:

c - collection to be checked for containment in this collection

Returns:

true if this collection contains all of the elements in the specified collection

See Also:

`Collection.contains(Object)`

removeAll

```
public final boolean removeAll(Collection<?> c)
```

Description copied from interface: `Collection`

Removes all of this collection's elements that are also contained in the specified collection (optional operation). After this call returns, this collection will contain no elements in common with the specified collection.

Specified by:

`removeAll` in interface `Collection<E>`

Parameters:

c - collection containing elements to be removed from this collection

Returns:

true if this collection changed as a result of the call

See Also:

Collection.remove(Object), Collection.contains(Object)

retainAll

```
public final boolean retainAll(Collection<?> c)
```

Description copied from interface: Collection

Retains only the elements in this collection that are contained in the specified collection (optional operation). In other words, removes from this collection all of its elements that are not contained in the specified collection.

Specified by:

retainAll in interface Collection<E>

Parameters:

c - collection containing elements to be retained in this collection

Returns:

true if this collection changed as a result of the call

See Also:

Collection.remove(Object), Collection.contains(Object)

Submit a bug or feature

For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.

Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms. Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.